# Project 1.1 — RepairDesk-Device-Repair-Chat-And-Ticket-Intake-Assistant

**A practice twin of Project 1 — SupportDesk.** Same skills, a new real-world job. **There are no steps here on purpose.** Read the story, build the whole thing yourself, and only open the hints or the solution if you get stuck.

**Type:** Multi-agent (2 agents + a router) · **Stack:** Python, OpenAI API (raw SDK), Pydantic · **Level:** Beginner-friendly

> **Stuck, or want to check your work?** Full hints · Complete solution

## The Story — what you are going to build

Picture a small shop that fixes phones, laptops and tablets. All day, people walk up to the front desk and say things. Some just ask questions — "what are your hours?", "how much is a battery?". Others describe a problem — "my phone screen is cracked, I need it today".

You are going to build **RepairDesk**: the front desk, done by AI. It does two jobs, and it must know which job a message needs:

- **Answer questions** like a friendly receptionist (hours, prices, services). This is the **Concierge**.
- **Take in a repair** — turn a messy, free-text problem like *"my laptop won't turn on, tried charging overnight"* into a clean, filled-in **repair ticket**: who the customer is (if they said), which device, how urgent, and a short summary. This is the **Intake** agent.

In the middle sits a **Router**. Before anything else, it reads the message and decides: is this a question (send it to the Concierge) or a repair (send it to Intake)? The person never has to pick a menu option — the Router picks for them.

**A real example to build and test against:**

- *"what are your repair hours?"* → the Router sends it to the **Concierge**, which answers in plain words.
- *"my phone screen is cracked, fix it today"* → the Router sends it to **Intake**, which makes a ticket: device = phone, urgency = high.
- *"Hi, I'm Sara Khan. My laptop won't turn on."* → **Intake** makes a ticket with the name **Sara Khan**, device = laptop.

- *"my laptop won't turn on, tried charging overnight"* → a ticket, but the name is left **empty** — the customer never said their name, so the agent must **not invent one**.

That last point is the whole lesson of this project: the ticket must have a **fixed shape** (only real device types, only low/medium/high urgency, name can be empty), so the model can never hand back junk your code can't use.

## How it runs, from the moment you type (picture the whole machine)

Read this slowly once and the whole project should form in your head — you'll *see* how a message moves through it.

You start the app in the terminal. It checks your API key once at startup; if the key is missing or wrong, it says so in one line and stops — it never limps on without a working key. Otherwise it prints a greeting and waits.

You type a line and press enter. The app does **not** answer it directly. It first hands the line to the **Router**, which asks the model one tiny question — *"is this a general question, or a device problem?"* — and gets back a fixed little answer: an agent name ( `concierge` or `intake` ) plus a one-line reason. If the model's answer is unclear or breaks, the Router quietly defaults to the Concierge, so the app always has somewhere to send the message.

- If the answer is **concierge**, your line is added to the running conversation and the Concierge streams a reply word by word. Because the whole conversation is kept and resent each time, it remembers what you said earlier. Before each call the old turns are trimmed so a long chat can't overflow — but the first line (its instructions) is never dropped.
- If the answer is **intake**, your line goes to the Intake agent instead. It asks the model to read your text and return it *in the exact ticket shape* — device, urgency, a short summary, and a name only if you actually gave one. If the model's answer doesn't fit the shape, or it refuses, the agent hands back nothing and the app says "I couldn't make a ticket" rather than storing garbage. If it fits, the filled ticket is printed.

Then the loop comes back around and waits for your next line. A rate-limit or an over-long message each print one friendly line and the loop keeps going; `/quit` ends it. That's the entire machine: **read a message → route it → either chat or make a ticket → never crash.** Everything you build is just those four moving parts — the Router, the two agents, and the strict ticket shape — plus the small helpers (settings, logging, memory-trim, prompt files) that hold them together.

## What you are building (the pieces)

Build a terminal app. You type a message; it either answers or makes a ticket.

- A **chat** that really talks to the model, **remembers** the conversation (so "what's my name?" works after you said it), and **streams** the reply word by word.
- A **repair ticket** as a checked shape (Pydantic): `customer_name` (can be empty), `device` (phone / laptop / tablet / other), `urgency` (low / medium / high), `summary`. The model must answer in exactly this shape.
- An **Intake agent** that fills that ticket from free text — and leaves the name empty when it isn't given.
- A **Concierge agent** that answers general questions.
- A **Router** that reads each message and picks Concierge or Intake, with a one-line reason, and falls back to Concierge if it isn't sure.
- **Safe errors:** the app never crashes. A missing key stops it at startup with one clear line; a bad key, a rate limit, or a too-long message each print one friendly line and keep going (or exit cleanly).

**Inputs:** whatever the user types. **Outputs:** a streamed answer, or a printed repair ticket. **Rules:** the ticket shape is strict; the name is never invented; the app survives every common error.

## How to practice this

1. Read this story. **Don't open the solution yet.**
2. Plan it on paper: the two agents, the router, the ticket shape, and what happens on each kind of error.
3. Build it yourself. A good order: first a chat that works, then memory + streaming, then the Intake ticket, then the Router that ties it together — but it's your call.
4. Test it on the real example above: a question, a repair, a repair with a name, and a repair with no name.
5. **Only if stuck**, read the hints. **To compare**, read the complete solution.

## Why this project is useful (read this at the end)

This is the smallest real shape of almost every AI product at a real job: **take a messy human message, decide what kind it is, and turn it into something your code can act on.** A support desk, a clinic front desk, a bank helpline, an IT helpdesk — they are all this: route the message, and pull a clean, fixed-shape record out of free text. If you can build RepairDesk from this story alone, you can build any of them. And the "never invent a value, keep a strict shape" habit is what keeps an AI feature from quietly filling your database with garbage.

## Done when

- A question is answered by the Concierge; a repair becomes a ticket from Intake; the Router picks correctly.
- The chat remembers earlier turns, and the reply streams live.
- A repair with a stated name fills the name; a repair with no name leaves it empty (never invented).
- A missing or bad key, a rate limit, and a too-long message are all handled with one clear line — no crash.
- Your tests pass: the settings, the history trim, the ticket/route shapes (no key needed), and a live run of the agents.

---

The full build, every file, is in the complete solution — use it to check your work, not to skip the thinking.